

Acadia Technical Reference Manual

William Kalfus

Version 9.0.0, March 2026

Contents

I	Introduction	3
1	Selected Specifications	3
2	Motivation in quantum control	5
II	Hardware	7
3	Logic design	7
3.1	Sequencer	7
3.1.1	Instruction fetch	7
3.1.2	Instruction format	8
3.1.3	Datapath resources	8
3.1.4	Bus	9
3.1.5	Bitfield tables	9
3.2	Sequencer bus decoder	9
3.3	Cache memory	9
3.4	Real-time DMA modules	13
3.4.1	DMA status signals	14
3.4.2	Output streams	14
3.4.3	Basic incrementing stream command	15
3.5	Primary memory interconnect	15
3.6	Stream processing path	15
3.6.1	DataMovers and DataMover controllers	15
3.6.2	Direct-to-memory streaming	15
3.6.3	Complex multiplier with accumulator (CMACC)	15
3.7	Data converter real-time configuration registers	17

4	Integrated hardware	17
4.1	Digital-to-analog converters	17
4.1.1	Gearbox FIFO	17
4.1.2	Interpolation	18
4.1.3	Numerically-controlled oscillator	19
4.1.4	Multi-band crossbar	19
4.1.5	DAC core	19
4.2	Analog-to-digital converters (ADCs)	19
4.2.1	ADC core	19
4.2.2	Multi-band crossbar	19
4.2.3	Numerically-controlled oscillator	19
4.2.4	Decimation	19
4.3	Clock Distribution	19
4.4	External Memory	19
5	Processing Subsystem (PS)	19
5.1	Interaction with the FPGA	19
III	Software	20
6	Building from source	20
7	Setting up a firmware distribution server	20

Part I

Introduction

Acadia is a laboratory instrument designed for carrying out complex sequences of signal synthesis and capture. It couples a very simplistic real-time sequencer to a general-purpose high-performance processor and orchestrates them in concert, reaping the benefits of each: the sequencer provides real-time sequencing with conditional control flow with very low latency, while the processor can carry out advanced computation to influence the sequencer’s behavior and rapidly generate new sequences. The sequencer’s simple nature allows sequence branching to have very low overhead, and because waveform synthesis is always scheduled dynamically by the sequencer, sequences can easily incorporate signal synthesis conditioned on measurements, computation results, memory values, and more. Captured signals pass through a reconfigurable network of processing units that are optimized for different aspects of signal processing such as wide-bandwidth capture, low-latency filtering, and high-precision spectroscopy. A full assembler and compiler are provided that allow complex programs involving the complete system to be expressed simply as Python classes. We also include a runtime manager that seamlessly deploys programs to remote hardware, transparently transfers and archives captured data, and performs arbitrary post-processing and visualization.

Acadia was designed with the challenges of quantum device research in mind, and a more thorough discussion of the associated considerations may be found in Section 2. However, its flexibility in orchestrating different signal processing functions allows Acadia to straightforwardly replicate the functionality of common lab instruments like oscilloscopes, signal generators, vector network analyzers, and spectrum analyzers. Along with a high-performance analog front-end, this can drastically lower the cost associated with having these capabilities available in a lab compared to commercial units, and the coexistence of these capabilities can allow bespoke combinations of functionality for unique experiments.

Acadia is targeted for implementation on a ZCU216 development board from Xilinx, though it is straightforwardly portable to other hardware. Many parameters of the design (such as memory depths) can be modified simply by editing a firmware definition file, but the default configuration has shown to be sufficient for many experiments in our lab. Some parameters of the default configuration include:

1 Selected Specifications

- A sequencer with single-cycle instruction decode and execution
 - 200 MHz clock rate
 - Two-cycle branch latency

- 4K-deep instruction memory, able to be reloaded in real-time from external memory
 - Eight 32-bit general-purpose registers
 - Eight 48-bit DSP units for real-time arithmetic
 - 32-bit memory bus for interface to the rest of the system
- 16 DACs operating at 6.4 GS/s, interpolated from an 800 MS/s fabric interface rate
 - DAC sample rates may be changed without firmware change to allow optimal placement of Nyquist zone boundaries
 - 32 KS memory for each DAC channel (approximately 40 μ s at the default rate), which can be reloaded by the sequencer in real-time from multiple external memory sources
 - A 48-bit numerically-controlled quadrature oscillator on each channel capable of modulating the samples at the full DAC bandwidth
- 16 ADCs operating at 2.4 GS/s, decimated to an 800 MS/s fabric interface rate
 - A 48-bit numerically-controlled quadrature oscillator on each channel capable of modulating the samples at the full ADC bandwidth
- All ADC outputs and two memory-to-stream units feed a processing/storage network containing configurable modules for concurrently processing streams of data
 - Two full-bandwidth capture paths write a stream directly to memory
 - Four complex multiplier/accumulator units
 - * Real-time signal filtering and decimation using an arbitrary filter kernel
 - * Short pipeline provides results to the sequencer within 50 ns
 - * Able to be automatically reset allowing back-to-back repeated captures
- A fast true-dual-port cache memory directly connected between the sequencer's bus and the PS AXI network
 - Organized as 32768 32-bit words
 - Optimized for very-low-latency communication between the sequencer and the processor (approximately 100 ns)
 - Allows fast real-time data access
- Two 4 GB banks of external DDR4 RAM
- A 256-bit AXI interconnect allowing all memory regions to be concurrently accessed by the stream processing units and the processor

2 Motivation in quantum control

Modern experiments in quantum device research demand signal synthesis and capture sequenced on nanosecond timescales. A non-trivial amount of computation may be needed for controlling the flow of the sequence, and the desire to condition signal synthesis on the results of prior measurements with very low latency precludes the use of commercial arbitrary waveform generators (AWGs). Furthermore, low tolerance for nondeterministic latencies or jitter while interacting with qubits requires that any processing logic be fully synchronous with waveform synthesis, making it difficult to integrate multiple instruments reliably.

The flexibility and ease of use sought from such an instrument makes interaction with high-level languages and standard processors desirable, but the complexity of modern CPUs' datapaths and the latencies added by typical communication interfaces make them ineffective real-time controllers. This latency can greatly exceed the length of a measurement sequence, so even for computations in between measurements, these latencies can drastically reduce the duty cycle with which measurements are performed.

Because of this, custom control hardware for quantum computers has been developed independently by multiple research groups. Most existing solutions implement a real-time processor in FPGA fabric which is capable of performing waveform parameter sweeps, control flow, and other specialized computations (such as random number generation or matrix multiplication). Analog-to-digital converters (ADCs) and digital-to-analog converters (DACs) are connected to the FPGA so that it can orchestrate the synthesis, capture, and processing of microwave signals interacting with the quantum hardware in real time. Often, the instrument will be configured and managed by an external computer and will be programmed to carry out a number of measurement sequences, recording data as necessary.

However, this approach requires the real-time processors to be capable of carrying out any computation that the complete measurement sequence could ever need, along with all of its variations. In view of the need to also complete calculations with very low latency, this requirement often leads to bloated instruction sets, short datapath pipelines, and monolithic designs. When implemented in reconfigurable FPGAs, the performance is often sub-optimal and difficult to improve.

Many previous implementations have augmented the real-time processor with custom accelerators for particular applications, such as parametric waveform synthesizers or randomized sequence generators. However, as quantum computing advances and experimentalists ask more of their control hardware, users aim to add capabilities to their instruments like automated calibration routines, real-time error syndrome decoding, and benchmarking methods. The monolithic nature of most implementations can require a significant development effort and force the developer to remove functionality in order to accommodate a bespoke feature, even after overcoming the substantial learning curve that high-performance FPGA development presents to a researcher inexperienced in

this practice.

To resolve the conflicting requirements of high performance and ease of use, herein we describe the Acadia system architecture, which is intended to take advantage of tight integration between the FPGA fabric and embedded processors available in modern programmable logic systems. By integrating these resources on the same chip, the latencies associated with launching communication signals off-chip and being translated to standardized protocols can be reduced or eliminated, and higher-level programs can interact with real-time sequences on reasonable timescales for quantum control. This allows the computational capabilities previously required of the real-time controller to be offloaded to the embedded processor, which can then be programmed in a high-level language more familiar to users.

In particular, we focus on the Xilinx RFSoc series of products, as they offer large regions of FPGA fabric and high frequency analog data converters on-chip alongside a quad-core ARM processor. Targeted at directly synthesizing signals for wireless communications, the data converters in the RFSoc are able to directly synthesize signals up to approximately 10 GHz, offering a promising opportunity to alleviate the restrictions imposed by conventional pulse synthesis techniques when scaling quantum processors. The harmonics and image tones introduced by mixing elements exacerbate the issue of qubit frequency crowding, while driving with high power distorts pulses and introduces spurious emissions, thereby increasing the difficulty of driving parametric processes which require signals with extremely stable phase and frequency relationship. This often requires signals to be mixed with one another to achieve sufficient syntonization, but because of the high power required for driving a mixer along with its conversion loss and spectral broadening, a significant amount of noise is added in this process and the high degree of connectivity required quickly makes this technique impractical for more than a handful of channels.

In contrast, the DACs and ADCs in the RFSoc have high enough analog bandwidths and sample rates so that pulses intended to drive transitions in superconducting circuits may be directly synthesized without the need for nonlinear upconversion processes driven by local oscillators. Additional signal processing capabilities in the datapaths of the mixed-signal tiles implement features that are desirable for quantum control (described below), alleviating requirements of the FPGA fabric.

Furthermore, the low (but nondeterministic) latency of the interconnect between the processors and the FPGA fabric allows one to offload computational tasks normally required of the FPGA to the ARM processor, such as generating new waveforms in response to parameter sweeps. The FPGA fabric then only requires a simple sequencer capable of real-time execution and conditional control flow, and programs running on the processors will have direct access to wave synthesis/capture memory and sequencer instruction memory.

These benefits (among others) motivate the implementation of a control system on the RFSoc, which we describe herein.

Part II

Hardware

Acadia targets hardware comprised of three subsystems:

- A region of FPGA fabric
- One or more processors that can boot Linux and interact with the FPGA over a memory-mapped bus
- An array of data converters (ADCs and DACs) that continuously stream data to/from the FPGA fabric

The following sections are dedicated to describing these components in further detail. This architecture is designed with the Xilinx RFSoc platform in mind, though this partitioning is very common in other systems that implement each of these discretely; therefore, porting to other platforms is straightforward.

3 Logic design

3.1 Sequencer

The sequencer is a small, single-cycle, deterministic CPU responsible for coordinating actions across the system. It steps through a sequence of instructions, each of which reads data from one or two data “sources” in the internal datapath of the sequencer and writes them to one or two “destination” ports. All source and destination ports are 32 bits wide.

3.1.1 Instruction fetch

The sequencer’s execution consists of sequentially fetching instructions from instruction memory and executing each one. One instruction is fetched and executed every cycle, and a pointer to the instruction currently being fetched is maintained in a register called the Program Counter (PC). The instruction memory has a two-cycle pipeline at its output, so the address of an instruction being executed will have occupied the PC two cycles prior, but the memory still emits (and therefore, the datapath processes) a new instruction every cycle.

An exception to this behavior occurs during branching. Because the instruction memory has a two-cycle pipeline, when the program requires the next instruction to be executed to not be the one retrieved two cycles prior, the sequencer must stall for two cycles while the required instruction is retrieved. This occurs automatically when a branch occurs; otherwise, the PC automatically increments each cycle with no stall. The PC is both a source and a destination, which allows both absolute and relative branching to be performed by writing to and reading from the PC. Conditional branching is implemented via a conditional write to the PC.

When the sequencer starts, it begins stepping through instruction memory at address 0 and executing instructions in order. The sequencer has only one instruction, which can either (a) retrieve values from any two source ports and write them into any two destination ports independently of one another, or (b) retrieve a value from one source and conditionally write it to one destination, conditioned on the value of an operation on a second source.

Many destination ports will take additional action when written to or provide access to logic units that carry out more complex operations, described in further detail below. This architecture greatly reduces the critical path of the sequencer, allowing the clock speed to remain reasonably high while allowing the sequencer to fetch and execute instructions with low latency. This behavior is extremely useful for distributed systems requiring a high degree of synchronicity, since this means that the sequencer is able to operate at the finest timebase resolution of the system.

3.1.2 Instruction format

In each instruction, two source ports (indicated by the instruction fields `src1` and `src2`)¹ are always read. A bitwise operation (specified by the instruction field `op_sel1`) is computed between the value of `src2` and the value of a special-purpose register referred to as the “mask”. The complete list of available operations are listed in Table 4. The result of this operation may optionally be used for the value of `src1`. For conditional writes, the result of the operation is always used as the test condition; the condition is said to pass if the value is non-zero. This behavior may be inverted by setting the `cond_inv` bit in the instruction.

3.1.3 Datapath resources

The sequencer embeds eight 32-bit general-purpose registers in its datapath, which may be interpreted as a 32-wide, 8-deep combinational memory with two read ports and two write ports. Therefore, in a single cycle, any two registers may be read from and any two registers may be written to, completely independent of one another. A value written to a register may be read from it in the next cycle. When reading from a register, the source multiplexer allows the register to be interpreted as a full 32-bit value or as a concatenation of two 16-bit values (which are then zero-extended to 32 bits on the left). This allows efficient packing of data when only small values are needed. Because these two 16-bit registers are aliases for the upper and lower halves of a register, writing to the register will affect the value read from both 16-bit aliases.

The sequencer also embeds eight DSP units in its datapath, which may be used to compute various arithmetic operations. The DSP unit has two input

¹We used the terms `src1` and `src2` to refer both to the 32-bit values being read as well as the 7-bit field in the instruction that determines which source ports are read; context should indicate which is meant.

registers (called “AB” and “C”), a result register (called “P”), and a configuration register; all but the configuration register may be used as inputs in the operation carried out by the unit, along with the P register of the neighboring unit. Each DSP unit also has an “enable” signal; when the enable signal is driven high, the the operation specified in the configuration register is carried out and stored into P. The enable signal may be set high indefinitely, cleared indefinitely, or pulsed for one cycle when writing to the configuration register. The enable signal may also be pulsed during any instruction by setting the appropriate bitfield; see Table 1. Setting the enable signal high indefinitely allows the operation to be recomputed each cycle without intervention from the sequencer; since the P register can be an input to the operation, this allows one to implement operations that automatically update, such as a self-incrementing counter. The AB and C registers are exposed to the sequencer as destination ports, and P is exposed as a source port (after passing through a single-stage pipeline delay).

An eight-deep stack is also embedded in the sequencer datapath. Writing to the stack destination “pushes” a new value to its top, and reading from its source “pops” a value from the top. Writing to a full stack has no effect, and reading from an empty stack produces undefined data.

3.1.4 Bus

The sequencer communicates with external peripherals through the use of a 32-bit bidirectional data bus. The bus is addressed by writing a value to the “bus address” destination. A write command may then be issued to the bus by writing to the “bus data” destination of the datapath, and a read command may be issued by reading from the “bus data” source. When writing to the bus, the bus address and bus data destinations may be written concurrently, allowing a complete bus write operation in one instruction. The bus address destination is a register, so repeated writes to/reads from the same address on the bus do not require the bus address destination to be written multiple times.

The way in which one should use the bus is determined solely by the modules connected to it. There is only one bus port on the sequencer, so to connect to multiple slave modules, an interconnect is necessary; see Section 3.2 for a description of Acadia’s implementation of a low-latency, single-master bus interconnect.

3.1.5 Bitfield tables

3.2 Sequencer bus decoder

3.3 Cache memory

The cache is a small, dual-port, on-chip synchronous memory located within the FPGA fabric. Unlike the other memories in the system, it’s not connected to the primary memory interconnect (described in Section 3.5); instead, one port is connected directly to the processor’s bus (described in Section ??) and the

Table 1: Sequencer instruction bitfields

Bits	Name	Description
127-113	Reserved	Reserved
112	cond	When this bit is set, the instruction is interpreted as a conditional write; otherwise, it performs two concurrent writes.
111-105	Reserved	Reserved
104	push_ret	When this bit is set, the value of the Program Counter minus 1 is pushed to the stack. Note that if the instruction specifies the stack as a destination, then that transfer will override this bit.
103-96	src1	For conditional writes, this source will be used to retrieve the value that will be written if the condition passes. For unconditional writes, this source will be used to retrieve the value that will be written to the destination specified by dest1 .
95-88	src2	For conditional writes, this source will be used to retrieve the value that will be tested in order to compute the condition. For unconditional writes, this source will be used to retrieve the value that will be written to the destination specified by dest2 .
87-80	dest1	Specifies the destination to which the value from src1 will be written in unconditional writes, or in conditional writes when the condition passes.
79-72	dest2	Specifies the destination to which the value from src2 will be written in unconditional writes. This is unused in conditional writes.
71	cond_inv	For conditional writes, the condition flag is inverted; in other words, when this bit is cleared, the condition passes when the result of the operation between src2 and the mask is nonzero. When this bit is set, the condition passes when it is equal to zero.
70-68	op_sel	Specifies the operation acting on src2 .
67	dsp_cep_en	When this bit is set, the DSP unit specified by the value of the dsp_cep field will have its enable signal pulsed.
66-64	dsp_cep	Specifies which DSP will have its enable signal pulsed if dsp_cep_en is set high.
63-32	imm1	An immediate value which may be used for src1 .
31-0	imm2	An immediate value which may be used for src2 .

Table 2: Available sources for **src1** and **src2**

srcx[7:3]	Name	Description
0000	Register	The 32-bit values stored in a register specified by srcx[2:0]
0001	Register (Low)	The lower 16 bits of the register specified by srcx[2:0] . The upper 16 bits are set to zero.
0010	Register (High)	The upper 16 bits of the register specified by srcx[2:0] . These 16 bits are shifted down to occupy the lower 16 bits of the data to be written, and the upper 16 bits are set to zero.
0011	Operation	The result of the bitwise operation between src2 and the mask.
0100	PC	The current value of the program counter
0101	Immediate	For src1 , this yields the imm1 field of the instruction. For src2 , this yields the imm2 field of the instruction.
0110	External	External digital signal directly available on the sequencer interface.
0111	Stack	The value at the top of the stack, which is then popped.
1000	Bus data	The data present at the read port of the bus.
1001	Reserved	Reserved; read as zero.
1010	DSP P	The value of the P register for a DSP slice. The DSP slice to be read is specified by srcx[2:0]
others	Reserved	Reserved; read as zero.

Table 3: Available destinations for **dest1** and **dest2**

destx [7:3]	Name	Description
0000	None	No write is performed
0001	Register	The 32-bit values stored in a register specified by destx [2:0]
0010	PC	The program counter. Note that writing to this destination will force the sequencer to stall for two cycles to account for instruction fetch latency.
0011	Mask	The mask register, to be used as the second operand for bitwise operations with src2 .
0100	External	External digital signals at the interface of the sequencer.
0101	Stack	Pushes a new value to the top of the stack. Note that if the stack is full, the write is discarded and the stack is unaffected.
0110	Bus data	Issues a write command on the bus to the address stored in the bus address register.
0111	Bus address	The bus address register.
1000	DSP config	The configuration register for the DSP slice specified by destx [2:0]
1001	DSP AB	The AB register for the DSP slice specified by destx [2:0]
1010	DSP C	The C register for the DSP slice specified by destx [2:0]
others	None	No effect

Table 4: Available operations specified by **op_sel**

op_sel	Operation	Notes
000	src2	No operation; src2 is passed through unmodified.
001	not src2	Bitwise inversion
010	src2 xor mask	When cond_inv = 1, the condition will pass when src2 = mask .
011	(not src2) xor mask	
100	src2 and mask	When cond_inv = 0, the condition will pass when any bits that are set in the mask are also set in src2 , ignoring any additional bits in src2 which may be set.
101	(not src2) and mask	When cond_inv = 1, the condition will pass when all bits that are set in the mask are also set in src2 , ignoring any additional bits in src2 which may be set.
others	Reserved	Undefined behavior; do not use.

other port is connected to the sequencer’s bus decoder (described in Section 3.2). This allows very-low-latency communication between them, which can enable the processor to influence sequencer behavior in real time.

The cache is the most direct way to exchange information that isn’t known at compile time between the processor and the sequencer. This allows one to write programs with variables whose values change between executions of the sequencer (and which would therefore not be integrated into the sequencer’s program as a constant).

The cache also provides a direct way for the sequencer to store values for the processor that didn’t pass through the stream processing path (described in Section 3.6). Common examples include values of sequencer registers and DSP units, or values retrieved from other modules on the bus. This can be especially helpful for debugging, as writing to the cache has very low overhead.

3.4 Real-time DMA modules

The DACs and ADCs in most mixed-signal systems accept or provide samples continuously, regardless of whether their controller has samples available or space to store them. This is usually handled by providing zeros to a DAC when the controller doesn’t want anything to be synthesized and discarding ADC values when they are undesired or unable to be received. In Acadia, it is the job of the direct memory access (DMA) modules to implement this behavior.

The DMAs are essentially programmable counters. Given a continuous stream of data, the DMAs create status signals and associate them with the stream so that downstream logic can decide whether or not to ignore the stream (the `valid` signal) and whether or not a given point in the stream should be considered the final piece of valid data (the `last` signal). The DMAs decide the values of these signals according to a command issued to it via its register interface.

Acadia implements a DMA for each DAC channel and each ADC channel. There is a register before the physical interface to each DAC so that only zeros are streamed when the channel’s DMA does not have its `valid` signal set. When `valid` is set, the register will instead pass the output of sample memory to the DACs. Similarly, the stream processing path (described in) will only accept data from an ADC when its DMA has its `valid` signal set; otherwise, the ADC data are discarded.

A command issued to the DMA determines its behavior. The DMA command interface has a 32-deep FIFO, and once it is triggered, the DMA will execute commands from the FIFO in order without no gaps between commands. Gaps are eliminated by retrieving the next command while one is being executed; this imposes a three-cycle minimum length for every command. If a command is issued to the DMA while it is running, there must be at least three cycles remaining in the current command’s execution before the next one begins if they are to be executed with no delay in between them. The FIFO command interface combined with the back-to-back command execution allows

the sequencer to “queue” multiple commands to create cycle-precise sequences of DAC streams and ADC captures.

3.4.1 DMA status signals

The DMA provides a handful of signals that control and reflect the status of its execution independent of the signals it attaches to the output streams.

The **trigger** signal instructs the DMA to begin reading and executing commands from its FIFO. As discussed above, there is a three-cycle command fetch latency that occurs before a command is executed; this imposes a three-cycle minimum latency between issuing the first command to a DMA and the assertion of **trigger**. If this is satisfied, then the DMA will begin executing its first command in the cycle following the assertion of **trigger**. This signal is particularly useful for aligning multiple DMAs, since their **trigger** inputs may be asserted simultaneously, thereby aligning their execution with single-cycle precision.

The DMA provides an externally-facing **running** signal to indicate when commands are being executed (and correspondingly, to indicate when it has completed all of its commands). This signal will be set when the DMA is triggered, and it will deassert once the final command in the FIFO has finished executing. This is also useful for synchronizing multiple DMAs, since the deassertion of **running** means that the DMA will need to be triggered again to continue, which can be done simultaneously for multiple DMAs.

The DMA also exposes a **flag** signal whose value is embedded in the command itself. This allows the DMA to indicate that it has reached a particular command without requiring the sequencer to keep track of it. This feature was developed to allow the DMA to actuate an RF switch during a command, but may be used for many other applications as well.

3.4.2 Output streams

The DMA has two output streams: a data stream and an address stream. Both interfaces have **valid** and **last** signals, which are direct copies of one another; this duplication is implemented so that the two streams are individually AXI-Stream compliant. The data stream simply attaches these signals to an incoming continuous data stream provided to the DMA on a separate port. The address stream reflects the value of an internal address register, whose behavior is determined by the history of commands issued to the DMA.

The data stream is intended to attach status signals to the continuous data produced by ADCs; ADC output data enter the DMA on the separate data input port, which are then reproduced on the data stream output along with associated **valid** and **last** signals. Downstream logic that expects **valid** and **last** signals (such as blocks with standard AXI-Stream interfaces) can then connect directly to this port, and the DMA controls which data they should accept.

The address stream is intended to be used with DACs; it can be connected directly to the address signals of the read port for DAC sample memory so that commands programmatically index the sample memory. The sample memory data output can then be connected to the DAC after “gating” it with the `valid` signal.

The DMA implements four different commands:

3.4.3 Basic incrementing stream command

This command specifies an address and a length. When the command is executed, the internal address register is immediately loaded with the provided address. The address register then increments by 1 every cycle, and

the number of cycles for which `valid` is set. During the last cycle of valid data, the `last` signal is set.

3.5 Primary memory interconnect

3.6 Stream processing path

3.6.1 DataMovers and DataMover controllers

3.6.2 Direct-to-memory streaming

3.6.3 Complex multiplier with accumulator (CMACC)

The Complex Multiplier and Accumulator (CMACC) module provides a low-latency solution for filtering and decimating signals in real time. Each sample that enters the CMACC is multiplied with the corresponding sample of an arbitrary complex kernel/window function, and the result is iteratively summed (“accumulated”) in a register (the “accumulator”). By choosing an appropriate window function, different bands of noise can be rejected from the incoming signal. This architecture can also allow the signal to be processed with a matched filter, which optimizes the noise rejection when trying to distinguish between known signals.

The internal datapath of the CMACC is a four-stage pipeline; a new set of samples is accepted every cycle and the accumulator reflects new inputs after a latency of four cycles. The stages, each of which performs multiple simultaneous operations, are as follows:

1.
 - Four complex samples enter the CMACC (each comprised of two 16-bit signed quadratures) which are summed together to produce a complex value with 18-bit signed quadratures.
 - The value of the kernel corresponding to the current value of the kernel pointer is produced at the output of the kernel memory.
 - If the kernel pointer has reached the value of the kernel end address register, it resets to the value of the kernel start address register. Otherwise, it increments by one.

2.
 - The quadratures of the kernel are multiplied pairwise with the quadratures of the summed sample. The resulting products are 34 bits wide.
 - The 34-bit terms are sign-extended to 47 bits and appropriately added and subtracted to produce a complete complex multiplication.
3. According to the values of the CMACC configuration register, the appropriate update operation for the accumulator is carried out according to the status flags associated with the product (e.g., whether it is the first or last sample of the kernel, or whether the input sample was noted as being last).
4.
 - According to the values of the CMACC configuration register, the accumulator latch may be updated. If it is, the CMACC status register will denote that the latch value is valid.
 - According to the values of the CMACC configuration register, a new value may be pushed into the output stream FIFO.

The CMACC module has a register interface that can be controlled by the sequencer via its bus. Using this interface, the sequencer can program the CMACC to produce different kinds of outputs, select different kernels, retrieve accumulator values, and generally control the behavior of the CMACC.

Each CMACC has a dedicated bank of kernel memory with a depth of 2048 samples. Before streaming a signal into the CMACC, the sequencer should specify the start and end addresses of the kernel in memory to the CMACC via a write to its register interface. These addresses are arbitrary and can be easily changed at will by the sequencer, allowing one CMACC to switch between multiple different kernels in real time.

An internal pointer register is used to step through the kernel memory as new samples enter the CMACC so that the samples can be multiplied by the appropriate value of the kernel. The pointer is initialized to the pre-programmed start address via a register write and it increments for every valid sample that enters the CMACC. When it reaches the pre-programmed end address, it resets to the starting address. If valid samples continue to enter the CMACC, the pointer will continue to increment again without any additional latency or dropped data; this enables the use of periodic kernels without needing to occupy more than a single period's worth of kernel memory. A common special case is the "boxcar" kernel, which only requires loading a single entry in kernel memory. This feature can also be used for decimation, since the CMACC can be configured to emit a valid output upon reaching the kernel end address (see below). The sequencer can also manually reset the kernel pointer to its start address via a write to the configuration register, which is required after updating the kernel start and end addresses.

The CMACC does not require consecutive samples to be valid. When samples enter the CMACC, if they are marked as not valid, they will not contribute to the accumulator and the kernel pointer will not increase. If an input sample is marked as "last", the CMACC can be configured to reset the accumulator

and/or produce an output; see below for a description of accumulator configuration and output behavior.

3.7 Data converter real-time configuration registers

4 Integrated hardware

Acadia’s FPGA design interacts with digital-to-analog converters (DACs) and analog-to-digital converters (ADCs) as well as the infrastructure necessary to support them (such as clocking and configuration). The properties of the data converters are determined solely by the available hardware and can vary significantly. Acadia was designed to target the Xilinx RFSoc product line and the default configuration targets the XCZU49DR device, which may be found on the ZCU216 evaluation board. However, we emphasize that the FPGA design of Acadia is quite flexible and may be straightforwardly adapted to other platforms.

Regardless, it’s worthwhile to describe some of the hardware features of the XCZU49DR directly used by Acadia, since these features motivated design decisions and are key to developing an understanding of the system as a whole. These descriptions can also act as a guide to those aiming to port Acadia to other platforms, as they highlight the features that one might seek out when choosing new hardware and can provide insight on any modifications that may be required.

4.1 Digital-to-analog converters

The DAC region of the RF Data Converter tile embedded in the RFSoc contains much more than simply a DAC core; notably, a chain of pre-processing logic preceding the DAC core and a highly-connected clock synthesis and distribution network for the sample clock, both of which are reconfigurable in software. We’ll describe the various features of the components integrated in the hardware below.

4.1.1 Gearbox FIFO

Samples are provided to the DAC via a first-in-first-out (FIFO) queue. The FIFOs are reconfigurable and may be configured for various different throughputs depending on the programmed sampling rate of the DAC, the clock frequency of the logic feeding it, and any interpolation (described below). In the default firmware configuration, every DAC is configured to accept four samples per clock cycle, each of which is comprised of two 16-bit quadratures. The quadrature values are expressed as two’s-complement signed numbers; correspondingly, the maximum positive number is $2^{15} - 1$ and the minimum negative number is -2^{15} . Note that this asymmetry restricts the valid full-scale values to a range of $[-1, 1)$, where the open interval on the right corresponds to an offset of one bit.

4.1.2 Interpolation

The DAC cores are often operated at very high sample rates in order to provide flexibility in output signal frequency. However, many applications use waveforms with bandwidths that are much smaller than the sample rate, so it would be very wasteful to store the waveform at the full sample rate when a smaller rate would suffice, especially because on-chip memory is a scarce resource. These goals reconciled by storing samples at a lower rate in memory and interpolating between them to feed the DAC core at its full rate.

Interpolation can take many forms and is commonly used in data analysis, but one should note that many common types of interpolation (e.g., linear, cubic spline, etc.) will introduce additional spectral content outside the bandwidth of the original pulse and may require significant hardware resources to implement at high bandwidth. Fortunately, when a finite-bandwidth signal is sampled at a rate of at least twice its bandwidth, it can be perfectly reconstructed at any point in time with no loss of information or additional distortion (this is restatement of the Nyquist criterion). Furthermore, this optimal interpolation strategy is simple to implement in hardware; it consists of adding zeros in between the low-rate samples and low-pass filtering the resulting stream (for additional details about the mathematics of this operation, we recommend Ch. 10 of [?]). Therefore, the only price paid for performing this interpolation is a small amount of additional latency for the samples to pass through the interpolation logic.

The RFSoc RF tile implements a reconfigurable interpolator just after the gearbox FIFO, so that the logic in the FPGA can provide data at a reduced rate but allow all further processing to occur at the full bandwidth of the DAC core. The interpolator block operates as described above; it inserts a given number of zero-valued samples between each input sample (such that its output sample rate matches that of the DAC core) and filters the result. The interpolator offers a selection of interpolation factors, but for a fixed input sample rate (given by the product of FPGA clock rate and the number of samples provided per clock cycle), the allowed output sample rate is constrained by the maximum rate of the DAC.

In Acadia the logic clock and the tile interface width are taken to be fixed, but may be adjusted with custom firmware. The interpolation factor and the DAC core sample rate may be adjusted dynamically, allowing flexibility in placing Nyquist zone boundaries. The default logic clock rate is 200 MHz and four complex samples are provided each clock cycle, resulting in a tile input rate of 800 MS/s. The DAC core has a maximum sample rate of 9.8 GS/s, so the interpolation factors and resulting DAC sample rates that may be used are: 1 (800 MS/s), 2 (1.6 GS/s), 3 (2.4 GS/s), 4 (3.2 GS/s), 5 (4.0 GS/s), 6 (4.8 GS/s), 8 (6.4 GS/s), 10 (8.0 GS/s), 12 (9.6 GS/s). Note that using the NCOs at sample rates above 7 GS/s requires special consideration; see below.

- 4.1.3 Numerically-controlled oscillator
- 4.1.4 Multi-band crossbar
- 4.1.5 DAC core
- 4.2 Analog-to-digital converters (ADCs)
 - 4.2.1 ADC core
 - 4.2.2 Multi-band crossbar
 - 4.2.3 Numerically-controlled oscillator
 - 4.2.4 Decimation
- 4.3 Clock Distribution
- 4.4 External Memory
- 5 Processing Subsystem (PS)
 - 5.1 Interaction with the FPGA

Part III

Software

6 Building from source

7 Setting up a firmware distribution server